

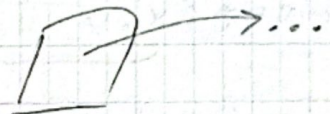
19.03.24г

Член-функции

```
struct Test {           // големина на Test = 1
    void g() {...}      → g(Test * const)
    void f() const      → f(const Test * const)
```

→ Няма да променя член-ранките на Test

```
Test {
    char * str;
    void f() const {
        ? str[0] = 'A'; // не го правим
    }
}
```



```
f(const Test & t)
    t.f(); // можем
    t.g(); // не можем, грешка при компилатора
// Ако нямаше & пак не можем да извикаме g()
```

Конструктор → извиква се задължително при създаване на обекта.

→ инициализира променливите
→ отваря / затваря външни ресурси
// инициализира памет

Test() → дефолтен конструктор
// важен е за масивите

Деструктор → може да е само една

→ освобождава динамичната памет /
затваря външни ресурси

// test се освобождава сам

Ако нямаме отпечатване в конструктор и деструктор.
Test *p = new Test[3];
delete p;

Отговор: Това е undefined behavior
Ще отпечата 3 пъти конструктор. // 3x Test()
и 1 път деструктор. // ~Test()

Конструктор може да извиква друг конструктор.

Point {x, y;
Point (int x, int y) : x(x), y(y)
Point () : Point (0, 0)

Point
x=0;
y=0;
Point (int x, int y) ...
~~Point () {};~~ → Point () = default; // ще
// може да използваме
// оптимизации това, когато
// сме ги инициализираме

Конвертиращ конструктор

за версии
≤ C++ 11

Конструктор, който приема точно 1 параметър

x {
x(int a) — конвер. констр.
};
f(x obj) f(7) — създава временен x

$g(\text{const } X \&)$ // позволява да приемат lvalue и rvalue
 $g(78)$ // това е валидно // rvalue

$t(X \& \text{ref})$ // отаква обект // lvalue
 $t(7)$ // няма да се компилира

explicit $X(\text{int } a)$ // пишем explicit само на функции с 1 параметър

този конструктор не е конвертиращ

// Ако подадем $t(7)$ няма да се компилира,
ще трябва да подадем обект от X на функцията

Конструктори и деструктори при композиция

$X \{$
 $A \text{ obj1};$
 $B \text{ obj2};$
 $C \text{ obj3};$
 $X() \dots$
}

⚠ Конструкторът на X е
отговорен за създаването на
 A , B и C .

Конструкторът на X трябва да
каже, кои конструктори да
им се извикат

$X(\text{int } a): A(\text{int}) \quad B(\text{int}) \quad C(\text{int})$
 $\text{obj1}(a), \text{obj2}(a+1), \text{obj3}(a+2) \{ \}$
// X извиква тези конструктори

Ако в конструктора не е казано кой конструктор
да се извика, се извиква default конструктор,
ако няма default, няма да се компилира.

$x()$: $A()$ $B(int)$ $C(int)$
 $obj2(\text{3})$, $obj3(\text{4})$

// подредбата зависи от структурата
 Редът на извикване на конструкторите не зависи от инициализиращия списък, а зависи от инициализирането на структурата

struct X {

A obj1;

B obj2;

C obj3;

}

// конструкторите се извикват в този ред

X(): ...

{

после се изпълнява това

A, B, C

Ако не сложим нищо, ще са default

за теория

default
конс-
к-
ри-
те
им

} // отпегатва се първо A(), B() и C() и
 накрая X()

// Ако няма default конструктори за A() и B()
 ще няма да се компилира

struct Y {

A obj;

Y() = default; // извиква default на A

Y(int x): Y(), obj(~~x~~) не можем

}

~~y(3)~~ // няма двусмислица и
 НЕ СЕ КОМПИЛИРА

Деструктори при композиция

$\sim X()$ $\{$ деструктора на X

// извиква деструкторите на A, B, C

cout << " $\sim X()$ "

$\{ \} \rightarrow$ тук ги извиква
 $\sim C(), \sim B(), \sim A();$

A
B
C

↓ конструктори

↑ деструктори

// поне рѝ има
зависимост и
затова са в този
ред констр. и дестр.

struct X {

A obj;

y() = default;

X obj; A(), B(), C(), X()

$\{ \} \sim X(), \sim C(), \sim B(), \sim A()$

X {

A

B

}

A {

α

β

}

{ X obj;

// ще се отпечатат $\alpha(), \beta(), A(), B(), X()$

$\{ \sim X(), \sim B(), \sim A(), \sim \beta(), \sim \alpha()$

struct Z {

B* ptr;

→ това са външни ресурси



C & get;

// дължини сме да дадем стойност

~Z() Не извиква деструктора на B, C

OOP принципи - 4 са

⚠ • Абстракция

- Използваме нещо без да се интересуваме как работи.
- Скриване на ненужните детайли.

• Капсулация:

- Ограничаване на достъпа.

→ Модификатори за достъп

- private - достъп в класа
- protected - достъп в класа и наследниците
- public - имаме достъп от всякъде

class/struct X {

private:

⋮

public:

⋮

}

struct A {

no default e

public

};

class B {

no default e

private

}

Уговорка: Големи обекти: class

Малки обекти: struct // пример Point

```
Time {  
private:  
    int hours; [0, 23] - валидни данни  
public:  
    // позволяваме достъп но под контрол
```

→ get и set функции

get() - връща копие / константна референция /
константен указател

set() - позволява промяна но под контрол

```
public:  
    int getHours() const { // константни са  
        return hours;     винаги  
    }
```

```
void setHours(int hours)  
{ валидни }
```

Уговорка: за get функции

- Когато говорим за примитивни типове ги връщаме като копие
- Когато говорим за обекти, връщаме константни референции.

```
const A& getA() const (...)
```


Задача: Клас човек, който има:

име : поне 2 символа, започва с Главна,
горини [5; 90] другите са малки
с дължина най-много 20 символа

Mutable

→ mutable член-данни

→ Те могат да бъдат променени дори от
константни функции

```
struct A {  
    mutable int x;  
    ✓ void f() const { x++; } // можем да променим x
```

```
}  
✓ g(const A& ref) // може  
    ref.x++;
```